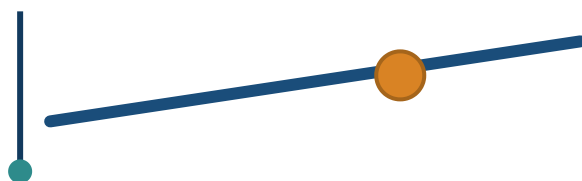


DOCUMENTACIÓN TÉCNICA

Ball-and-Beam

Control PID en tiempo real · STM32F401RE + FreeRTOS



HC-SR04 → Kalman → PID → Servo

Sistema de balanceo con filtro de Kalman e IPC por colas

Autor: Martín Sferco · 2026

Índice

1. Visión general del sistema
2. Configuración de hardware (CubeMX)
3. Arquitectura de software y estructura de carpetas
4. Pipeline de tiempo real (tasks, colas, semáforos)
5. Drivers de periféricos
6. Algoritmos: Kalman y PID
7. Wiring, tasks y glue
8. Flags de compilación
9. Decisiones de diseño e historial

1. Visión general del sistema

Un objeto desliza sobre una **barra articulada** que un servo inclina. El objetivo es **posicionar el objeto** en un punto elegido de la barra y mantenerlo estable, pese a perturbaciones.

La distancia del objeto se mide con un sensor ultrasónico, se **filtra** (Kalman) para quitar ruido, un controlador **PID** calcula cuánto inclinar la barra, y un **servo** ejecuta esa inclinación. El punto deseado (*setpoint*) lo fija el usuario con un **potenciómetro**.

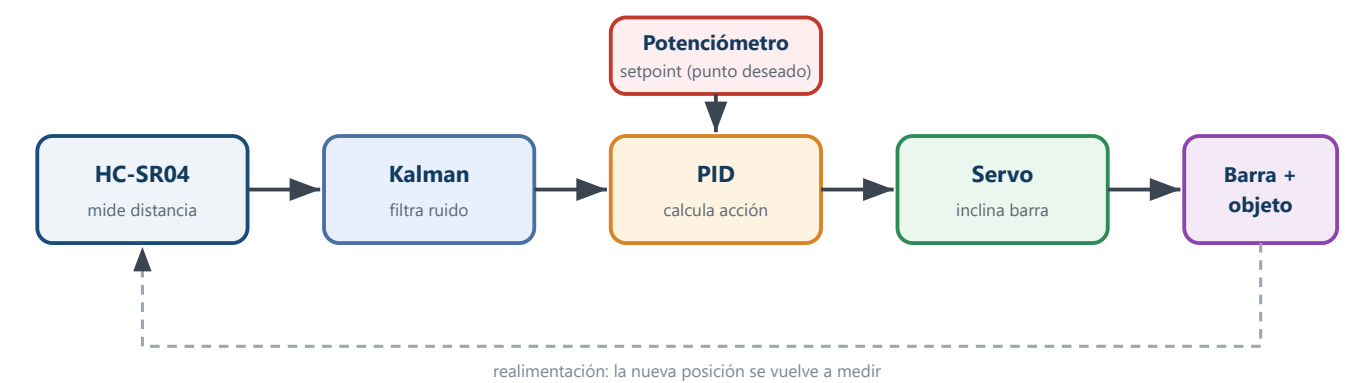


Figura 1 — Lazo de control cerrado del ball-and-beam.

Ítem	Detalle
Plataforma	STM32F401RE (NUCLEO-F401RE), Cortex-M4F
Framework	HAL (STM32CubeMX) + FreeRTOS (API nativa)
Sensores/actuadores	HC-SR04 (ultrasonido), servo MG90S (PWM), potenciómetro (ADC)
Idea central	Todo el código de app en App/ → sobrevive a la regeneración de CubeMX

2. Configuración de hardware (CubeMX)

2.1 Árbol de reloj

Oscilador **HSI** (16 MHz interno), **sin PLL** — mismo estilo que el proyecto de referencia **BlinkyRTOS**. SYSCLK = HSI directo = **16 MHz**, AHB /1 → **HCLK = 16 MHz**. Como TIM2/TIM3/TIM4 están en APB1 (prescaler ≠ 1), el HAL les aplica ×2, por lo que **corren a 16 MHz**. Todos los prescalers de timer se calculan sobre 16 MHz (PSC=15 → 1 µs/tick).

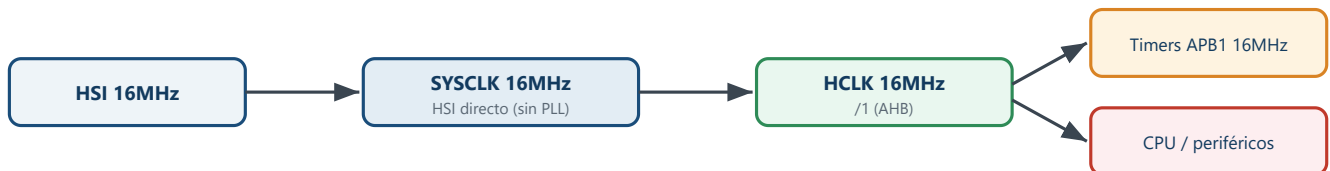


Figura 2 — Cadena de reloj (HSI directo, sin PLL). Los timers APB1 quedan a 16 MHz.

2.2 Periféricos y pines

Periférico	Uso	Configuración	Pin(es)
TIM2	Input Capture del ECHO	PSC=15 (1µs/tick), 32-bit, NVIC prio 5	PA0 / A0 (CH1)
GPIO	TRIG del sensor	Output push-pull	PA9 / D8
TIM3	PWM del servo	PSC=15, ARR=19999 (50 Hz)	PA6 / D12 (CH1)
TIM4	Tick de 100 ms	PSC=1599, ARR=999, NVIC prio 5	— (sin pin)
ADC1	Potenciómetro	12 bits, IN4, polling, sampling 480, sin DMA/IRQ	PA4 / A2
USART2	Consola / traza	115200 8N1 (COM virtual ST-Link)	PA2/PA3
TIM5	Time base del HAL	(automático de CubeMX)	—
LD2	Heartbeat del lazo	GPIO Output (LED onboard)	PA5

¿Por qué prioridad NVIC = 5? `configMAX_SYSCALL_INTERRUPT_PRIORITY = 5`. Una ISR que llama `...FromISR()` (dar un semáforo) debe tener prioridad NVIC numéricamente ≥ 5. TIM2 (echo) y TIM4 (tick) dan semáforos → van en **5**. Con prioridad menor, **hard fault**.

2.3 FreeRTOS

- API nativa** de FreeRTOS: tasks con `xTaskCreate`, arranque con `vTaskStartScheduler()` (estilo BlinkyRTOS). CubeMX genera la capa CMSIS-RTOS v2 pero **no se usa**: el arranque `osKernel*` se reemplazó por `vTaskStartScheduler()` en `USER CODE WHILE` (re-aplicar tras regenerar CubeMX).
- `configTOTAL_HEAP_SIZE = 24576` · `configMAX_PRIORITIES = 56` · `configMINIMAL_STACK_SIZE = 128`.
- `configUSE_QUEUE_SETS = 1` → agregado a mano en `USER CODE` de `FreeRTOSConfig.h` (CubeMX no lo expone; así sobrevive a regeneraciones).

3. Arquitectura de software

Regla de oro: **todo el código de aplicación vive en App/** (que CubeMX no toca); `main.c` / `freertos.c` quedan solo con *glue* mínimo dentro de secciones `USER CODE`. Después de cada `Generate Code` se verifica que `App/` sigue en el build y el glue quedó intacto.



intacto (sin tocar) HAL / drivers generados `tim.c` · `adc.c` · `usart.c` · `gpio.c` `stm32f4xx_it.c` (TIM4_IRQHandler)

Figura 3 — Separación App/ (propio) vs Core/ (CubeMX). El puente es `app.h`.

Estructura de carpetas

```
PIDController/
├─ App/          ← TODO el código de aplicación (source folder en .cproject)
│  ├─ Inc/      app.h app_config.h kalman.h pid.h selftest.h
│  │           hc_sr04.h servo_mg90s.h potentiometer.h
│  │           task_sensor.h task_kalman.h task_pid.h task_motor.h task_pot.h
│  └─ Src/      (los .c correspondientes)
├─ Core/        ← CubeMX (glue en USER CODE)
├─ Drivers/ Middlewares/ ← HAL + FreeRTOS (generado)
└─ PIDController.ioc
```

4. Pipeline de tiempo real

El sistema son **5 tareas** conectadas por **4 colas** (profundidad 1), un **queue set** en el PID y **2 semáforos binarios** disparados desde ISRs.

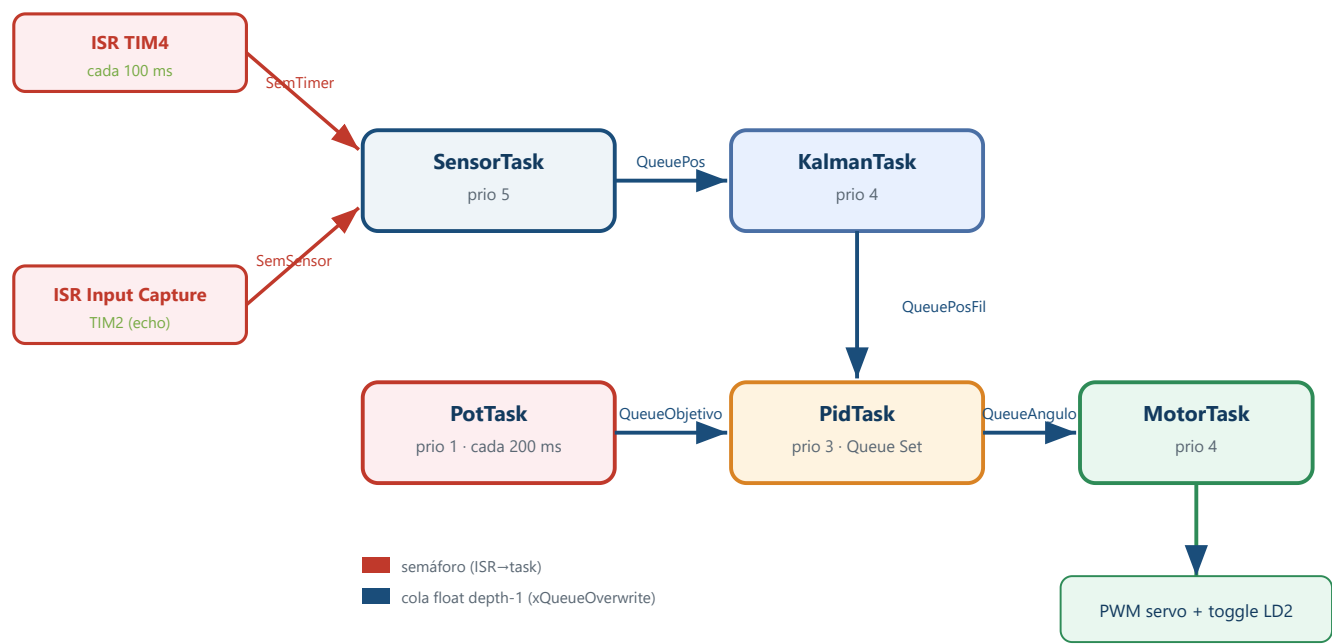


Figura 4 — Pipeline completo: ISRs, 5 tasks, colas y queue set.

Task	Prio	Espera / recibe	Produce
SensorTask	5	SemTimer (+ SemSensor)	QueuePos
KalmanTask	4	QueuePos	QueuePosFil
PidTask	3	QueueSet{QueuePosFil, QueueObjetivo}	QueueAngulo
MotorTask	4	QueueAngulo	PWM servo + LD2
PotTask	1	temporizada (200 ms)	QueueObjetivo

Cronología de un ciclo (100 ms)

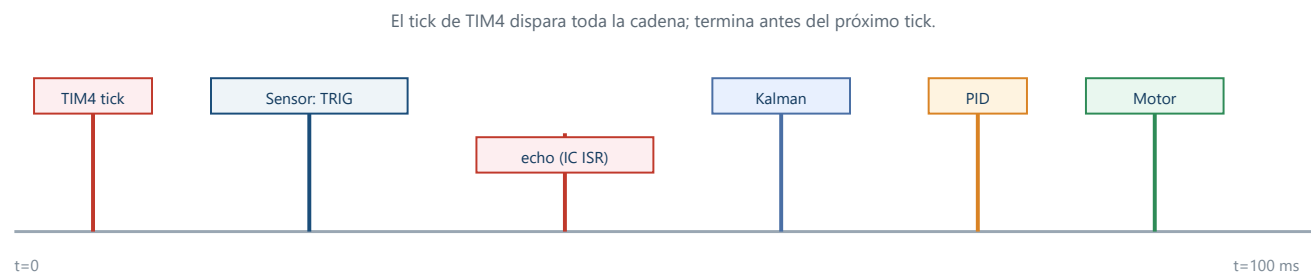


Figura 5 — Secuencia temporal disparada por el tick de 100 ms.

Las colas son `float` de **profundidad 1** con `xQueueOverwrite` : siempre gana el dato más reciente (control en tiempo real, no encolar históricos). El **queue set** deja al PID bloquear en dos fuentes (posición filtrada y setpoint) con una sola

primitiva.

5. Drivers de periféricos

Los tres son **multi-instancia** y **RTOS-agnósticos** (solo HAL).

5.1 `hc_sr04` — sensor ultrasónico (Input Capture, por interrupción)

- `HC_SR04_Init(h, htim, ch, trig_port, trig_pin)` — configura y registra la instancia.
- `HC_SR04_SetRange / SetCompleteCallback` — rango válido y hook de fin de medición.
- `HC_SR04_Trigger(h)` — pulso de 10 μs + arma captura (único busy-wait, inofensivo).
- `HC_SR04_GetDistance(h,&cm)` — no bloqueante: OK / BUSY / TIMEOUT / INVALID.
- `HC_SR04_HandleInterrupt(htim)` — dispatcher desde el callback global del HAL.

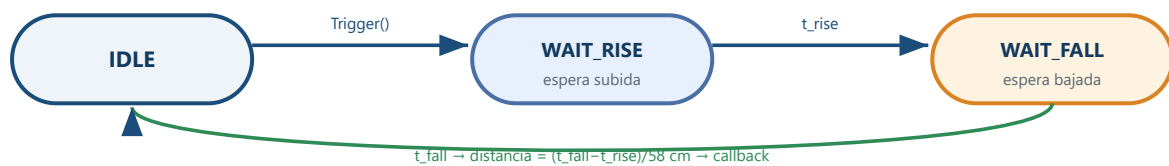


Figura 6 — Máquina de estados del HC-SR04 (una medición completa).

5.2 `servo_mg90s` — servo por PWM

Con el timer a 1 $\mu\text{s}/\text{tick}$ y $\text{ARR}=19999$, el registro CCR (en cuentas) **equivale al ancho de pulso en μs** . API: `Servo_Init`, `Servo_SetLimits` (calibración), `Servo_SetPulseUs`, `Servo_SetAngle` (mapea grados $\rightarrow \mu\text{s}$ y clampea), `Servo_GetPulseUs`.

5.3 `potentiometer` — potenciómetro por ADC (polling)

Lee sin DMA ni IRQ (baja frecuencia). API: `Pot_Init` (full_scale 4095 = 12 bits), `Pot_ReadRaw`, `Pot_ReadPosition_cm` (mapea crudo $\rightarrow \text{cm}$ en $[0, \text{range_cm}]$).

6. Algoritmos: Kalman y PID

6.1 Filtro de Kalman (2 estados)

Estado $x = [\text{posición}, \text{velocidad}]$, modelo de **velocidad constante**, paso dt fijo (0.1 s). Suaviza el ruido del HC-SR04 estimando también la velocidad.

Matriz	Valor	Rol
F	$\begin{bmatrix} 1, dt \\ 0, 1 \end{bmatrix}$	transición (posición avanza con la velocidad)
H	$[1, 0]$	medición (solo se observa la posición)
Q	$q \cdot \begin{bmatrix} dt^3/3, dt^2/2 \\ dt^2/2, dt \end{bmatrix}$	ruido de proceso ($1q = \text{más responsivo}$)
R	escalar (~ 0.09)	varianza de medición del sensor

Ciclo por muestra: **predicción** ($x=Fx$, $P=FPF^T+Q$) + **corrección** con la ganancia de Kalman. API: `Kalman_Init`, `Kalman_Update(z)` (devuelve la posición estimada), `Kalman_Reset`.

6.2 Controlador PID

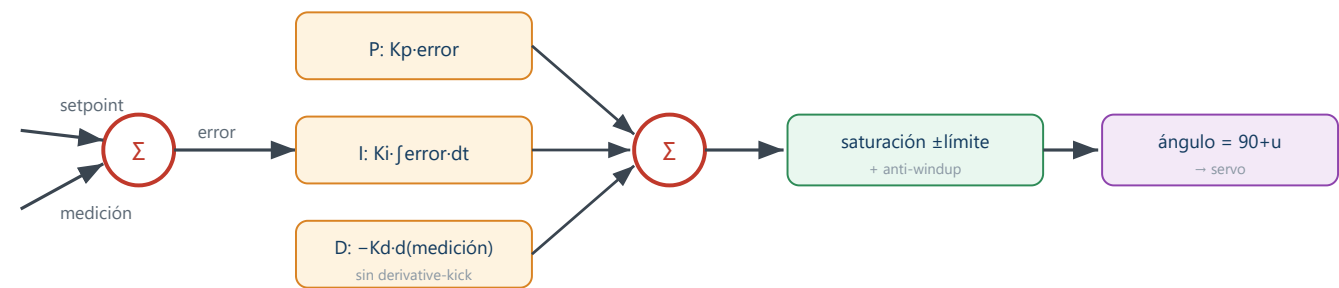


Figura 7 — Estructura del PID: derivada sobre la medición, anti-windup y saturación.

- **Derivada sobre la medición** (no el error) → sin *derivative-kick* al mover el setpoint.
- **Anti-windup** por integración condicional (*clamping*): si la salida satura y el error empuja más hacia la saturación, congela el integrador.
- **Saturación** a `[out_min, out_max]` ($\pm 20^\circ$ alrededor del centro).
- API: `PID_Init`, `PID_SetLimits`, `PID_Compute(sp, meas)`, `PID_Reset`.

7. Wiring, tasks y glue

7.1 `app.c / app.h` — capa de wiring

Es lo único que `main.c` necesita conocer. Contiene las instancias globales (`g_sensor` , `g_servo` , `g_pot`), los objetos de IPC (semáforos, colas, queue set) y los hooks de ISR.

- `App_Init()` (desde `USER CODE 2`): crea semáforos y colas (chequeo NULL → `Error_Handler`), arma el queue set, `Pot_Init`, arranca `HAL_TIM_Base_Start_IT(&htim4)` y crea las 5 tasks.
- `App_OnSensorComplete_FromISR` / `App_OnTimerTick_FromISR` — dan los semáforos.
- `__io_putchar` — retarget de `printf` a USART2.
- `App_LogTrace` — imprime `z`, `fil`, `sp`, `u`, `ang` (formato `%d.%03d`, sin `%f`).

7.2 `app_config.h` — el único lugar para tunear

Stacks y prioridades de las tasks, rangos del sensor/pote, ganancias de Kalman y PID, calibración del servo y los flags de compilación. **Tunear el sistema = editar este archivo.**

7.3 Glue en `Core/`

```
// main.c — USER CODE Includes
#include "app.h"

// main.c — USER CODE 2
App_Init();

// main.c — USER CODE 4
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim){ HC_SR04_HandleInterrupt(htim); }

// main.c — HAL_TIM_PeriodElapsedCallback, USER CODE Callback 1
if (htim->Instance == TIM4) { App_OnTimerTick_FromISR(); }

// main.c — USER CODE BEGIN WHILE (arranque nativo, estilo BlinkyRTOS)
vTaskStartScheduler(); // reemplaza osKernelInitialize/MX_FREERTOS_Init/osKernelStart

// FreeRTOSConfig.h — USER CODE Defines
#define configUSE_QUEUE_SETS 1
```

La ISR `TIM4_IRQHandler` (en `stm32f4xx_it.c`, generada por CubeMX) llama `HAL_TIM_IRQHandler(&htim4)`, que a su vez dispara nuestro `HAL_TIM_PeriodElapsedCallback`.

8. Flags de compilación (`app_config.h`)

Flag	Producción	Qué hace en 1
<code>APP_RUN_SELFTESTS</code>	0	Corre <code>SelfTest_Run()</code> al arrancar (PASS/FAIL por UART) antes de crear el lazo.
<code>APP_USE_SYNTHETIC_SENSOR</code>	0	Reemplaza el sensor por una señal sintética y el pote por un setpoint fijo — valida la cadena sin hardware.
<code>APP_LOG_ENABLED</code>	0	Imprime la traza <code>z</code> , <code>fil</code> , <code>sp</code> , <code>u</code> , <code>ang</code> por UART (subir para el tuning; bajar en producción por la latencia del UART).

Self-tests (módulo `selftest`)

Todo el `.c` está dentro de `#if APP_RUN_SELFTESTS` (con el flag en 0 no aporta código). Casos: convergencia de Kalman, atenuación de outlier, seguimiento de rampa, R grande vs chico; y en PID: solo-P exacto, saturación, sin derivative-kick, anti-windup vs sin AW, y un lazo simulado que converge al setpoint.

9. Decisiones de diseño e historial

Por qué está hecho así

Decisión	Motivo
Todo en <code>App/</code>	Robustez ante regeneración de CubeMX (premisa central)
API nativa de FreeRTOS, <code>freertos.c</code> intacto	Estilo del proyecto de referencia
Colas depth-1 + <code>xQueueOverwrite</code>	Tiempo real: siempre el dato más nuevo, no encolar históricos
Queue set (no mutex/flags)	El PID espera dos eventos con una sola primitiva bloqueante
Módulos puros Kalman/PID	Testeables (y compilables en host si hubiera gcc)
Tests detrás de flag	Validar sin ensuciar el binario de producción
<code>%d.%03d</code> en vez de <code>%f</code>	newlib-nano no imprime floats por defecto
NVIC prio 5 en TIM2/TIM4	Requisito de FreeRTOS para <code>...FromISR()</code>

Historial de construcción (git)

1. `chore:` baseline + driver servo
2. `refactor:` mover app a `App/` y glue mínimo en `main.c`
3. `feat:` CubeMX ADC1 (pote), TIM4 100ms, heap y queue sets
4. `feat:` módulos puros Kalman y PID + self-tests (opt-in)
5. `feat:` pipeline FreeRTOS (5 tasks, 4 colas, queue set, semáforos) + smoke-test
6. `feat:` lazo cerrado real + tuning inicial
7. `chore:` ajuste de stacks, gating de logs, verificación post-regeneración
8. `fix:` setpoint fijo en modo sensor sintético
9. `merge:` arquitectura completa (Etapas 0–6, validada por smoke test)

Estado: software completo, compila, arranca y validado por el smoke test sintético. Pendiente solo el montaje físico + tuning — ver **GUIA_CONTINUACION_HARDWARE.md**.